

ALT4Decompile: Inferring C-aligned Abstract Loop Tree for LLM-Based Binary Decompilation

Yongpan Wang, Puzhuo Liu, Xin Xu, Siyuan Li, Yaowen Zheng, Xiaodong Gu, and Beijun Shen

Abstract—Decompilation refers to the process of recovering high-level (C) language code from low-level (assembly) code. Recent Large Language Model (LLM)-based methods can generate re-executable code but struggle with the enormous syntax gap between assembly and C languages. Loop structures in C are fragmented into complex jumps within assembly languages, which poses a challenge to LLMs decompiling assembly language, which is mainly pre-trained on C.

To bridge this gap, we propose ALT4Decompile, which refactors assembly to closely resemble C language structure, thereby preventing hallucinations in LLMs caused by complex jumps. Specifically, we construct a C-aligned Abstract Loop Tree (ALT) based on specific jump patterns of Assembly that explicitly aggregates fragmented Assembly blocks into corresponding high-level loop structures (e.g., nested loops). Finally, we fine-tune an LLM adapted to ALT to generate decompiled code and then improve its output by fixing specific errors and restoring symbols. Evaluated against 12 baselines (including rule-based SAILR and LLM-based LLM4Decompile) on Decompile-Eval, MBPP, and ExeBench, ALT4Decompile achieves state-of-the-art correct re-executable results (even under four commonly used obfuscation techniques): i.e., a 70.4% test case pass rate on Decompile-Eval, a 10.6% improvement over prior best work. Furthermore, analyses on real-world software and a user study further show its practicability in understanding binary functions. The datasets and source code used in this paper are available at <https://github.com/wang-yongpan/ALT4Decompile>.

Index Terms—Binary decompilation, large language models, program analysis, control flow graph, reverse engineering.

I. INTRODUCTION

Decompilation aims to recover C code equivalent to a given binary executable, playing a crucial role in various reverse engineering tasks such as vulnerability discovery [20], [29], [39], [46], malware analysis [18], [34], [58], [64], and understanding or reusing closed-source software [27], [33], [42], [47]. However, this task is inherently challenging due to irreversible information loss during the compilation from C to assembly languages [35], [53], [66], such as non-intuitive code transformations introduced by compiler optimizations (e.g., the loss of C structures) [21], [28], [57] and the loss of

readability-oriented features (e.g., variable names and function signatures) [32], [61].

Directly generating recompilable and re-executable code can integrate potential dynamic analysis and testing methods [14], [16], [60], thereby reducing the workload of manual reverse. Traditional decompilers, such as IDA Pro [8] and Ghidra [6], primarily focus on translating Assembly code into pseudo-code to assist reverse engineers in understanding program semantics. However, although pseudo-code offers better readability than Assembly code, its *non-executable nature limits the application of decompilation results in downstream analysis*, such as code repackaging [49], platform migration [16], [56], and vulnerability debugging [14], [16].

Existing approaches still show limitations in terms of automation, reliability, and accuracy in generating recompilable and re-executable code. Specifically, existing approaches could be classified into three categories: **Rule-based methods** such as Ghidra [6] and SAILR [13] generally employ static and dynamic analysis to recover readable pseudo-code, yet their effectiveness heavily relies on domain-specific expertise and manual verification. **Refinement-based methods** use large language models (LLMs) to refine the pseudo-code generated by rule-based methods to reduce manual work and improve readability or correctness. LLM4Decompile-Ref [51], DecLLM [60], and ReySm [61] focus on refining the output of existing decompilers (e.g., Ghidra). However, such methods are susceptible to decompiler-induced binary execution logic distortion [22], [67] and require considerable cost to adapt to new architectures [6], [8]. **End-to-end methods** address the above limitations by processing Assembly code directly. Nova [36] fine-tuned DeepSeekCoder [26] using hierarchical attention. LLM4Decompile-End [51] trains an end-to-end decompilation model on a large-scale Assembly-C pairs dataset from ExeBench [11]. SccDec [24] enhances it by introducing a self-constructed context from compilation. These methods show great potential, but still face recompilable and re-executable problems.

Specifically, existing methods simply treat Assembly as a high-level language, relying entirely on LLMs' inherent capabilities while ignoring the critical syntax gap between high- and low-level languages. During compilation, loop structures in C are fragmented into complex instruction jumps. This fragmentation challenges LLMs that are primarily pre-trained on C languages. Although control flow graphs (CFGs) provide detailed jump flows, the loop structure of C is invariably fragmented across multiple basic blocks of CFG. LLMs struggle to grasp these scattered relationships to correctly reconstruct structures like loops. Consequently, complex control flows fre-

Yongpan Wang, Xiaodong Gu, and Beijun Shen are with Shanghai Jiao Tong University, Shanghai, China. E-mail: {frankile, xiaodong.gu, bjshen}@sjtu.edu.cn.

Puzhuo Liu is with Tsinghua University, Beijing, China. E-mail: liupz@mail.tsinghua.edu.cn.

Xin Xu is with The Hong Kong University of Science and Technology, Hong Kong SAR, China. E-mail: xxuca@connect.ust.hk.

Siyuan Li is with the School of Cyber Science and Technology, Shandong University, Jinan, China. E-mail: siyuan@sdu.edu.cn.

Yaowen Zheng is with the Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China. E-mail: zhengyaowen@iie.ac.cn.

Corresponding author: Xiaodong Gu. E-mail: xiaodong.gu@sjtu.edu.cn.

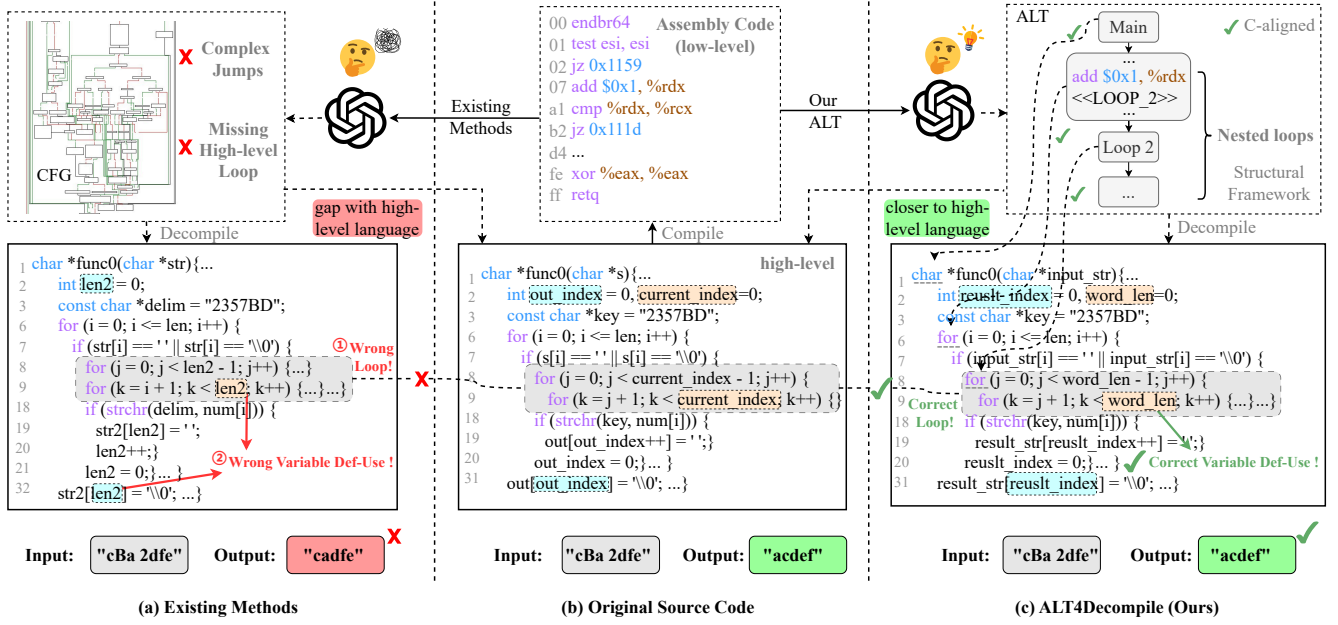


Fig. 1: A Motivation Example from Decompile-Eval. IN means instruction normalization.

quently induce structure and variable errors (shown in ① and ② of Figure 1(a)), leading to execution behavior deviations.

In this paper, we present ALT4Decompile, a novel LLM-based binary decompilation technique that refactors Assembly into a C-aligned **Abstract Loop Tree** (ALT, Section III-B). For complex jumps of assembly, ALT initially constructs a **structural framework from the C perspective** rather than relying solely on LLMs' inherent capabilities. Specifically, the loop structure of C languages and some specific jump patterns of assembly languages share common logic features (e.g., repeated executions indicating loops). By analyzing specific jump patterns, we aggregate instructions from the same C loop into unified, hierarchically organized nodes. These nodes explicitly define loop boundaries aligned with C structure syntax, successfully mitigating LLMs' struggles with complex control flows (e.g., on the Decompile-Eval benchmark, our improvement increases from 22.8% on samples without nested loops to 26.4% on samples with nested loops). This aggregation and layering mechanism enables LLMs to initially identify the program structure among complex jumps, thereby preventing disorientation and reducing hallucinations. To generate decompiled code, we fine-tune an LLM adapted to ALT and improve its output by fixing specific errors and restoring symbolic information.

We evaluate ALT4Decompile on three public datasets (Decompile-Eval, MBPP, Exebench) with input/output (I/O) samples and compare ALT4Decompile against three categories of baselines, i.e., general-purpose LLMs, rule-based decompilers, and LLM-based decompilers (including refinement-based and end-to-end methods). The results show that ALT4Decompile achieves new state-of-the-art (SOTA) performance. For example, on the Decompile-Eval dataset, ALT4Decompile achieves a recompilation rate of 96.8% (4% improvement over SOTA), a re-execution rate of 58.7%

(8.9% improvement over SOTA), and a test case pass rate of 70.4% (10.6% improvement over SOTA). Furthermore, ALT4Decompile consistently outperforms baselines across four common code obfuscation techniques, real-world software (e.g., WeChatWin.dll), and model types. A user survey further confirms that our decompilation output is the preferred choice over competing methods.

In summary, our contributions are as follows:

- To the best of our knowledge, ALT4Decompile is the first approach to bridge the syntactic gap (e.g., complex instruction-level jumps) between low- and high-level languages by leveraging their shared logic features, thereby improving LLM-based decompilation performance. And we release the model weights and code on the website [2] to support future research.
- We propose the first practical algorithm to construct ALT from Assembly code, demonstrating its feasibility and superiority in assisting LLM-based decompilation by establishing a structural framework from the high-level language perspective.
- We conduct extensive experiments (e.g., code obfuscation and real-world software) to assess the effectiveness of our method. The experimental results show that ALT4Decompile achieves new SOTA performance in binary decompilation, attaining a 70.4% test case pass rate on the Decompile-Eval dataset, representing a 10.6% improvement over the previous best approach.

II. MOTIVATION

Problem. Compared to traditional rule-based decompilers, LLM-based decompilers promise automatically executable outputs, supporting downstream reverse engineering. However, existing LLM solutions [24], [44], [51] conservatively treat Assembly as a new high-level language, ignoring its unique

low-level syntax. Unlike C, Assembly lacks explicit high-level loop structures, relying instead on complex jumps (e.g., `JMP <address>`) of instructions. LLMs, which are pretrained primarily on high-level language, struggle to reconstruct relationships between scattered instructions and translate them into corresponding loop structures of C.

Figure 1 illustrates it using some simplified Decompile-Eval [51] examples. The source code (Figure 1b) features a three-level nesting loop removing characters specified by the constant *key* and sorts the remainder (e.g., “cBa2dfe” becomes “acdef”). We use LLM4Decompile [51] (Figure 1a) for comparison, which is the first and most widely adopted approach. LLM4Decompile incorrectly generates a two-level loop and merges the definitions and usages of two distinct variables (*current_index*, *out_index*) into one (*len2*), causing failure. In our manual analysis, excluding failures caused by missing corresponding content such as data, over 80% of the failures arise from complex jumps induced by loops.

Insight. To tackle these problems, we must construct valuable information about C structures from Assembly to guide the LLM. Simply relying on LLMs’ inherent capability to infer C code execution logic from raw Assembly or CFGs is challenging, because LLMs are pretrained primarily on high-level language. However, converting Assembly to C-like pseudo-code scales poorly and risks execution logic distortion [22], [67]. Therefore, we aim to refactor Assembly into a structure resembling C while retaining original execution logic in the corresponding structure. Execution logic without complex jumps is easier for LLMs to process. Inspired by prior works [1], [6], [8], we observe that some specific jump patterns of Assembly code mirror C loop structures (i.e., repeat the execution of the same code segment). Based on this insight, we propose ALT (Section III-B), inferred directly from Assembly. ALT is a rooted tree $T = (N, E, r, \mathcal{I})$, where N represents loop structure nodes; $E \subseteq N \times N$ defines parent-child relationships (e.g., nested sub-loops); $r \in N$ is the function body root; and $\mathcal{I}$ maps each node to an ordered sequence of corresponding Assembly instructions.

Why ALT is effective? It equips LLMs with a high-level language structural framework inferred via simple shared logic features. Unlike CFG-based methods [44] forcing LLMs to implicitly infer C structures from scattered blocks, ALT explicitly aggregates instructions belonging to the same C loop into unified nodes. This establishes clear boundaries mirroring C structure syntax. As Figure 1c shows, every instruction that belongs to the same C loop structure is aggregated into a single node, thereby producing a well-structured, high-level logic representation with clear boundaries. This simplified structure effectively guides the LLM to achieve accurate loop structure recovery. Even when a small fraction of loop nodes with unidentifiable structural boundaries are aggregated, the internal jumps within these nodes remain simpler than the global control flow of the original code, which facilitates inference for LLMs. Notably, our empirical analysis shows that ALT’s relative improvement over the baseline **increases from 22.8% on samples without nested loops to 26.4% on samples with nested loops**. In addition, Table IV and Section IV-E1

show that the performance of our ALT (57.2% RE) is greater than CFG (51.8% RE) for decompilation. Unlike intermediate representations (IRs), ALT is exclusively designed to generate re-executable decompiled code.

III. DESIGN OF ALT4DECOMPILE

A. Overview

The core idea of our method is to construct the abstract high-level structures (i.e., nested loops) from low-level binary operations, which enables LLMs to better understand the relationships among scattered basic blocks. Inspired by prior works [1], [6], [8], we derive these structures by analyzing specific jump patterns, such as the repeated execution of a given code segment. Figure 2 illustrates the overall workflow of ALT4Decompile, comprising three key stages. First, we construct C-aligned abstract loop trees (ALT) from input binaries (Section III-B). Next, we fine-tune a language model (ALT4EXE) using the ALT to generate decompiled code (Section III-C). Finally, we refine the output by correcting predefined errors while recovering symbolic information (Section III-D).

B. Constructing C-Aligned Abstract Loop Tree

Binary code often contains complex jump instructions that disrupt statement coherence, impeding language models from accurately understanding high-level code logic. To overcome this, we design ALT, a novel abstract binary representation that leverages shared logic features between low- (Assembly) and high-level (C) languages. For instance, loop structures consistently manifest as cycles with back-edges in CFGs across both levels. Thus, ALT reconstructs the initial high-level loop framework directly from Assembly code.

Specifically, ALT is a tree structure encapsulating jump structures like loops. The root node represents the entire function body. At each jump target, a special *marker instruction* is inserted to indicate a jumping unit (Figure 3). The root’s child nodes represent first-level jumping units (e.g., Loop_0 and Loop_2 in Figure 3) within the corresponding high-level language, named after their corresponding markers. This parent-child relationship recursively descends to leaf nodes, which represent atomic, jump-free instruction blocks. Each node strictly orders its Assembly instructions by address. The formal definition of ALT is detailed in Section II. The recovery process of ALT consists of CFG extraction, instruction normalization, jumping unit identification, and tree construction.

1) *CFG Extraction*: We start by extracting the CFG from the binary code, which is widely used in binary analysis [62], [63]. Nodes of a CFG represent basic blocks composed of multiple Assembly instructions, while edges represent possible control flow transitions between these blocks. Although the CFG partially captures program logic [44], LLMs still fail to accurately model complex control flow, leading to unstable decompilation results (detailed in Section IV-E1).

2) *Instruction Normalization (IN)*: Differences in data storage mechanisms between low-level and high-level languages necessitate the extraction of isolated data segments based on their absolute addresses. Following this extraction, we

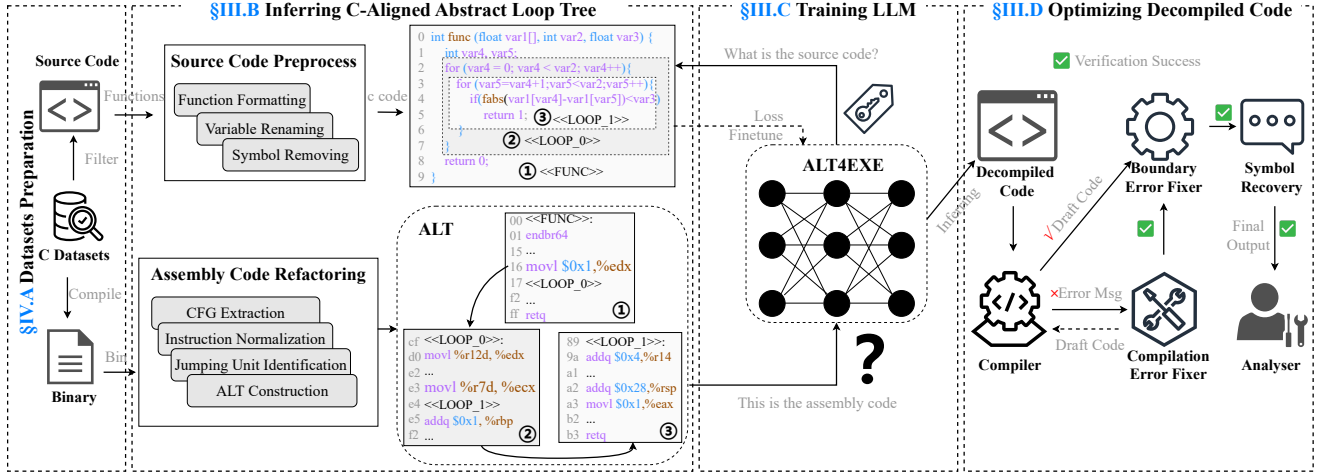


Fig. 2: The Workflow of ALT4Decompile.

normalize the instructions within the CFG basic blocks using two distinct approaches. First, for jump instructions (e.g., *je*), we convert absolute addresses to relative offsets from the function's entry point. Second, for instructions referencing data segments (e.g., *.data*, *.rodata*), we extract the corresponding content (i.e., convert them to strings) and append it to the instructions as comma-separated Assembly comments.

3) *Detect Jumping Units:* We identify jump units from the normalized CFGs and prioritize loops due to their structural significance in defining execution paths and affecting decompilation correctness, as Section II demonstrates. We defer the incorporation of other components, such as conditional structures, to future investigations. Loops consistently manifest as back edges in CFGs, directing control flow to repeat code fragments and forming strongly-connected subgraphs. Leveraging this structure [1], [6], [8], we identify loop blocks in the binary's CFG corresponding to high-level language loops and extract their nested hierarchies.

Algorithm 1: Jumping Unit Identification

Input: Control Flow Graph G

Output: Nested Loop Structures ls

```

1  $ls \leftarrow \emptyset$ ;
2 Function DetectLoop( $G$ ):
3    $sub\_gs \leftarrow \text{GetConnectedGraphs}(G)$ ;
4   while  $sub\_gs \neq \emptyset$  do
5      $g \leftarrow sub\_gs.pop()$ ;
6      $loop \leftarrow \text{CreateNewLoop}(g, ls)$ ;
7     for  $sub\_g \in sub\_gs$  do
8       if  $sub\_g \in g$  then
9          $nl \leftarrow \text{CreateNewLoop}(sub\_g, ls)$ ;
10         $loop.children.append(nl)$ ;
11         $ls = ls \cup \{nl\}$ 
12    $ls = ls \cup \{loop\}$ ;
13 return  $ls$ ;
14 end

```

Inspired by previous methods [1], [6], [8], Algorithm 1 outlines the process of detecting loop structures from CFGs, which is crucial for constructing ALT. The principle is to

detect strongly connected subgraphs and analyze their inclusion relationships to determine nesting. Notably, we add some measures to handle situations that fail to detect, e.g., selecting a minimum in-degree entry node for multi-entry subgraphs. First, we extract connected subgraphs via depth-first search (DFS) [54] (Line 3), where each subgraph corresponds to a c-aligned loop structure. We then traverse these subgraphs to build the nested loop structure ls (Lines 4-13). Specifically, we check if a subgraph fully contains another to identify lower-level loops (Line 8), while *CreateNewLoop* manages loop creation (Lines 6, 9). Finally, the algorithm outputs the unified nested loop hierarchy, matching the gray boxes in Figure 3(c) and (d).

4) *Tree Construction:* Next, we construct an abstract tree based on the identified nested loops, as described in Algorithm 2. The process begins by initializing the loop tree with a root block labeled the function's name or address. We then recursively traverse the CFG from the entry node to identify nodes belonging to the same loop block. For nodes within a loop structure (Line 4-16), we create a new logic block named $\langle\langle LOOP_index \rangle\rangle$ as the current block, where *index* increases with each new block. All nodes belonging to the loop are merged into this block, which is then added as a child node to the upper-level logic block (initially from the root). The block's name is inserted as a special instruction in the parent logic block to mark the loop's position. Sub-loops (Lines 12-13) and exit nodes (identified using the function *GetOutNodes*, Line 14) are processed recursively until all CFG nodes are processed. To address compiler optimizations that merge identical basic blocks, we deduplicate Assembly instructions based on their addresses to avoid redundancy. For each node outside a loop structure (Lines 17-25), we collect its Assembly instructions along with its list of child nodes. If a node ends with a call instruction (function *IsCall*, Line 22), it is merged with its child nodes (function *MergeNodes*, Line 28), and the instructions are added to the parent logic block in address order. This process is applied recursively to all descendant nodes. Finally, the algorithm outputs a C-aligned abstract logic tree, which is essential for training our

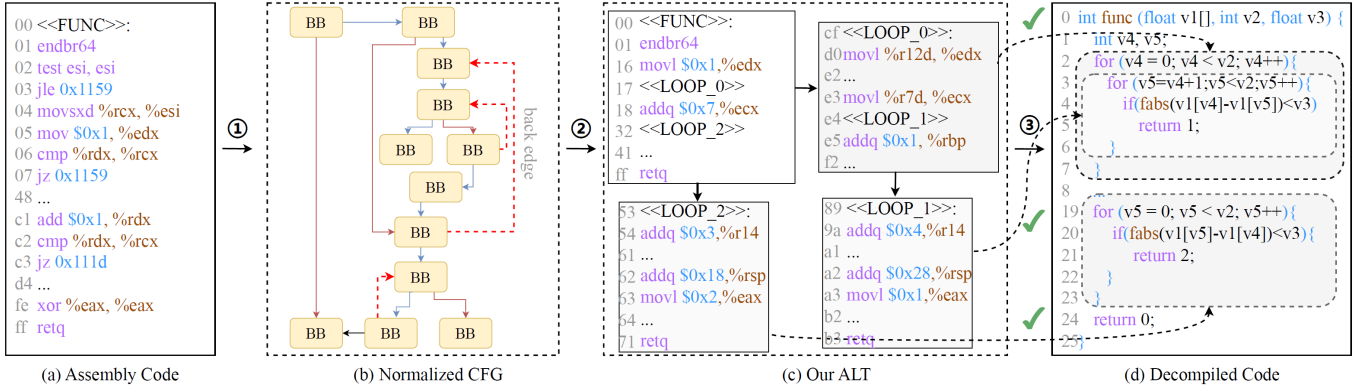


Fig. 3: The Process for Constructing ALT. Grey boxes represent the identified loop structures.

Algorithm 2: ALT Construction

Input: Control Flow Graph G , All loops ls , entry node en , parent block pb
Output: C-aligned Abstract Loop Tree alt

```

1   $alt \leftarrow \text{LoopBlock}(\text{func\_name}), \text{index} \leftarrow 0, pb \leftarrow \text{salt};$ 
2  Function ConstructALT ( $G, ls, en, pb$ ):
3       $loop \leftarrow \text{in\_loop}(en.\text{addr}, ls);$ 
4      if  $loop$  and not  $loop.\text{processed}$  then
5           $loop.\text{processed} \leftarrow \text{True};$ 
6           $block\_name \leftarrow \langle\langle LOOP\_index \rangle\rangle;$ 
7           $index \leftarrow index + 1;$ 
8           $lb \leftarrow \text{LoopBlock}(block\_name);$ 
9           $pb.\text{add\_children}(lb);$ 
10          $lb.\text{add\_ins}(G, en);$ 
11          $pb.\text{add\_ins}(block\_name, \text{NULL});$ 
12         for  $sl \in \text{loop.subloops}$  do
13              $alt \leftarrow \text{ConstructALT}(G, ls, sl.\text{node}, lb)$ 
14          $out\_nodes \leftarrow \text{GetOutNodes}(G, en, loop);$ 
15         for  $on \in out\_nodes$  do
16              $alt \leftarrow \text{ConstructALT}(G, ls, on, lb);$ 
17     else
18          $last\_ins \leftarrow \text{GetLastIns}(en);$ 
19          $succ\_ns \leftarrow \text{GetSuccessors}(en.\text{addr});$ 
20          $pb.\text{add\_ins}(G, en);$ 
21         for  $sn \in succ\_ns$  do
22             if  $IsCall(last\_ins)$  then
23                  $alt \leftarrow \text{MergeNodes}(G, ls, sn, pb)$ 
24             else
25                  $alt \leftarrow \text{ConstructALT}(G, ls, sn, pb)$ 
26     return  $alt;$ 
27 end

28 Function MergeNodes( $G, ls, en, pb$ ):
29      $pb.\text{add\_ins}(G, en);$ 
30      $succ\_ns \leftarrow \text{GetSuccessors}(en.\text{addr});$ 
31     for  $succ\_n \in succ\_ns$  do
32          $alt \leftarrow \text{ConstructALT}(G, ls, succ\_n, pb)$ 
33     return  $alt;$ 
34 end

```

decompilation model.

Figure 3 shows the ALT construction processing using a concrete example. For the given binary, we first disassemble it (Figure 3(a)) and extract its CFG. We then apply two normalization methods to the instructions within each basic block of the CFG. Following normalization, we identify back edges (highlighted as red dashed lines in Figure 3(b)) to infer the C-aligned loop structures. Finally, we construct the ALT (Figure 3 (c)), composed of loop blocks. The root block represents the entire function body, while the remaining blocks correspond to the identified loops. The hierarchical structure of the ALT reflects the nesting relationships among these loops (Figure 3 (d)).

C. Training Decompilation Model

The constructed ALT provides a logically coherent representation that is similar to the source code. Our subsequent objective is to train an LLM to generate the decompiled code using our ALT. To serialize ALT, node instructions are organized by address, with nodes themselves arranged in parent-child order. Details of the training dataset are shown in Section IV-A1. We adopt a sequence-to-sequence framework to train our decompilation LLM, namely ALT4EXE. Specifically, we initialize ALT4EXE using checkpoints from LLM4Decompile [51] and fine-tune it using the constructed pairs of (ALT, preprocessed Source Code). Following established practices in previous work [24], [36], [51], we employ the same training template in the training phase. In addition, we discard all function pairs that exceed the maximum input length of the model.

D. Optimizing Decompiled Code

Finally, we optimize the decompiled code generated by the trained ALT4EXE. The refinement process is designed primarily to ensure workflow completeness rather than to serve as a main contribution. As demonstrated in Table II, ALT outperforms other baselines even when this process is omitted. In contrast to existing refinement methods [59], [60], we incorporate a verification mechanism after each step to prevent excessive modifications. Specifically, we utilize an LLM to evaluate the differences between the input and output,

TABLE I: Prompt Templates for Components of ALT4Decompile

Component	Prompt Template
ALT4Decompile	This is the assembly code:\n {ALT } \n What is the source code?
Compilation Error Fixer	Please fix the following code based on the error messages provided by the GCC compiler to ensure successful compilation. The fix should minimize changes to the original code while ensuring the correctness of its logic.
Boundary Error Fixer	Analyze the following code to identify any possible boundary condition errors (The judgment statement of the loop is wrong, such as n is changed to $n-1$, the index overflow of the array is not reinitialized, such as $i++$ is not initialized to the original 0 in time, the variable i of the loop is initialized incorrectly, and so on), and ensure that the execution logic of the code is correct. If found, modify only the necessary parts and output the modified code. Do not modify any unrelated sections. No explanation.
Variable Name Recovery	Help me rename the variables for the snippet in the following C code. The renaming principle is: high readability, simple and easy to understand, and frequently used. No explanation.
Comments Generation	Help me add code comments for the code snippet in the following C code. The principle of adding comments is: high readability, simple and easy to understand, a simple code line does not add comments. No explanation.

determining whether the changes satisfy the modification requirements of that step. We discard the changes that fail to meet these criteria.

This involves refining its execution logic using compiler feedback and correcting predefined boundary errors. Additionally, we recover the variable names that are renamed as *var_i* during training from symbol recovery. These optimizations are performed using general-purpose LLMs, guided by the prompt templates outlined on the Table I.

1) *Compilation Error Fixer (CEF)*: We first compile the decompiled code using the *gcc* compiler. If compilation fails, the error messages along with the decompiled code are fed into the CEF for correction. The resulting code is then recompiled. If compilation errors persist, the fix process is repeated for up to three iterations. Once the compilation succeeds or the maximum number of attempts is reached, the code is directly forwarded to the next fixer.

2) *Boundary Error Fixer (BEF)*: Due to the division of logic blocks, a small number of execution errors arise from incorrect loop boundary conditions. To address this, we analyze common error patterns in ALT4EXE’s output and design predefined prompts to guide the BEF in correcting them. Typical issues include misconfigured loop termination condition (e.g., using $n+1$ instead of n), missing initialization of loop index variables (e.g., failing to set the index to 0), and so on. These types of errors can often be identified and corrected without requiring access to the source code.

3) *Symbol Recovery*: Following previous work [32], we restore symbolic information to enhance the readability of the decompiled code. Specifically, we focus on recovering two widely used types of symbolic information: variable names and comments. This restoration is achieved by leveraging off-the-shelf LLMs, which infer meaningful variable names and generate relevant comments based on the context and logic of the decompiled code.

IV. EVALUATION

We evaluate ALT4Decompile through automatic and human studies, and examine each component’s impact on overall performance. Specifically, we address the following research questions:

- **RQ1**: How effective is ALT4Decompile at decompilation compared to existing SOTA baselines?
- **RQ2**: Is ALT4Decompile still effective when handling binaries obfuscated using different techniques?
- **RQ3**: What is the contribution of each component to the overall performance (e.g., ALT)?
- **RQ4**: To what extent does ALT4Decompile assist with reverse real-world binary software (e.g., WeChatWin.dll)?
- **RQ5**: How much does it cost (i.e., time and money) to use ALT4Decompile for binary decompilation?

In addition, we conduct a user study to assess its practical utility for reverse engineering.

A. Datasets

1) *Training Dataset*: We construct our training dataset in three steps: **1) Source Code Selection**. Following prior work [24], [36], [51], we use the *train_real_compilable* subset of Exebench [11] (~700K C functions). To align with our objectives, we apply three filters: retaining functions with 5–500 non-blank lines, and ensuring successful GCC [5] compilation. Additionally, observing that many functions contain a high proportion of trivial assignment statements, we require a minimum structure-to-line ratio of 1:200 to filter out simple functions (while randomly keeping 20% of structure-free functions to ensure diversity). This yields a final set of ~40,000 C functions to balance training costs. **2) Source Code Pre-processing**. To help the ALT4EXE model focus on recovering execution logic, we strip extraneous elements. Specifically, we normalize formatting with *clang-format* [3], uniformly rename variables (e.g., *var1*, *var2*) using *Tree-sitter* [10], and remove comments and keywords like *inline*. **3) Compilation & Disassembly**. We compile the retained functions into binaries using *gcc-8.4.0* under four optimization levels (O0–O3), and strip all symbols for fairness, producing ~160,000 binary functions. We then disassemble them into AT&T syntax using *Capstone* (aligning with LLM4Decompile’s format). Finally, we extract ALT from the binary functions and pair them with their corresponding source code to fine-tune our LLM.

2) *Test Dataset*: Following prior work [24], [36], [51], [52] like Decompile-bench, we select three recognized datasets measuring functional correctness via input/output verification:

TABLE II: Effectiveness of ALT4Decompile against Baselines on the Three Decompile Datasets

Model	Decompile-Eval			MBPP			ExeBench		
	RE	RC	TCP	RE	RC	TCP	RE	RC	TCP
General-Purpose LLMs									
DeepSeek-V3	39.5	86.9	51.6	47.0	77.0	50.5	16.7	67.1	19.8
GPT-4o	19.2	81.1	34.1	23.0	74.5	26.6	15.9	65.2	18.8
Claude-3.5-Sonnet	50.5	96.0	63.8	48.0	80.5	53.2	19.2	75.1	20.5
o1-mini	18.3	78.4	33.7	18.5	63.5	23.8	14.4	64.1	17.2
Rule-Based Decompile Tools									
Hex-Rays	27.0	30.0	28.3	25.7	28.3	26.4	3.7	25.4	3.8
Ghidra	26.7	48.2	26.9	24.6	29.1	25.8	3.2	24.3	4.1
RetDec	32.9	79.1	40.4	35.6	72.0	39.1	19.9	51.6	21.1
SAILR	7.2	30.6	10.1	16.6	34.6	18.6	8.1	37.4	12.5
Refinement-Based Method									
LLM4Decompile-Ref	52.3	94.7	64.2	39.4	64.9	41.8	11.9	51.5	12.8
End-to-End Methods									
Nova-6.7B	34.1	85.7	45.4	20.0	65.0	23.2	10.1	60.2	11.4
LLM4Decompile-End-6.7B	46.7	88.0	57.9	46.5	78.3	50.6	21.5	78.3	24.4
ScdDec-6.7B	49.8	92.8	59.8	47.6	80.1	52.1	23.1	84.7	23.9
ALT4Decompile-6.7B	58.7	96.8	70.4	52.4	81.1	56.4	26.4	87.1	27.8
ALT4Decompile-ALT*	57.2	91.3	67.1	51.1	80.6	55.9	26.1	87.0	27.4

* means that ALT4Decompile removes all refinement processes of Section III-D.

1) **Decompile-Eval** [51]: Contains 164 C functions (from *HumanEval*), yielding 656 binaries across 00–03, evaluated via assertion-based tests (4–5 per function). 2) **MBPP** [52]: 974 Python programming problems ported to C. Compilation under four optimization levels yields 3896 binary functions, and each with 3 assertion tests. 3) **Exebench** [11]: Comprises real-world C functions sourced from GitHub. Specifically, we use its `test_real` subset, featuring complex user-defined data structures and richer dependencies. After filtering non-compilable functions, we retain 1,575 C functions (6,300 binaries across 00–03). In addition, we modify the test code by adding macro definitions to calculate the proportion of passed cases.

B. Experimental Setting

1) **Baselines**: We compare ALT4Decompile with three categories of baselines. 1) **General-purpose LLMs**: DeepSeek-V3 [43], GPT-4o [7], o1-mini [9], and Claude-3.5 [4], evaluated via official APIs given their massive parameter sizes. 2) **Rule-based Tools**: Three widely recognized top-tier decompilers in the field [23] (Hex-Rays [30], Ghidra [6], RetDec [40]), plus the recent compiler-aware method SAILR [13]. 3) **Dedicated LLM-based Methods**: Three end-to-end models (LLM4Decompile-End [51], Nova [36], ScdDec [24]) and one refinement-based model (LLM4Decompile-Ref [51] which refines Ghidra’s output). We use their official HuggingFace weights and default hyperparameters.

2) **Metrics & Implementation**: We evaluate decompilation performance using three metrics: 1) **Re-Execution Rate (RE)**: Verifies functional correctness by checking if the decompiled code passes all test cases. 2) **Re-Compilation Rate (RC)**: Assesses syntactic correctness by checking if the code successfully recompiles. 3) **Test Case Pass Rate (TCP)**: Measures the average proportion of passed test cases across all functions.

ALT4Decompile is implemented in ~2000 lines of Python. For disassembly and CFG extraction, we use Angr [1] purely for implementation convenience, as our method only requires standard binary analysis capabilities. We use a 6.7B parameter ALT4EXE model trained for one epoch (max length 4096, learning rate 5e-6, batch size 8, gradient accumulation 16) in ~8 hours on an Ubuntu 22.04 server with an Intel 48-core CPU and two Nvidia H100 80GB GPUs. DeepSeek-V3 is employed for code optimization.

C. Effectiveness of ALT4Decompile (RQ1)

In this RQ, we evaluate the effectiveness of ALT4Decompile against all baselines on three open-source datasets (Table II). Overall, it consistently outperforms all baselines across all metrics. On the Decompile-Eval dataset, for example, it achieves 58.7% in re-execution (RE), 96.8% in re-compilation (RC), and 70.4% in test case pass (TCP) rates. Compared to the state-of-the-art end-to-end method, ScdDec (49.8% RE, 92.8% RC, 59.8% TCP), ALT4Decompile improves RE, RC, and TCP by 8.9%, 4.0%, and 10.6%, respectively. Notably, the more significant gain in TCP than in RE highlights that ALT4Decompile generates higher-quality decompiled code with superior execution correctness and functional equivalence to the original source. Furthermore, we also demonstrate that even **without the refinement steps** (Section III-D) based on the general-purpose LLM, the performance is still superior to any of the current competing baselines (e.g., our ALT4Decompile-ALT is 57.2 RE in Table II on the Decompile-Eval dataset).

The results also reveal distinct trends among the baselines. Among general-purpose LLMs, Claude-3.5-sonnet exhibits impressive capabilities, trailing ours by only ~10% on Decompile-Eval, likely due to its advanced proficiency in resolving coding scenarios. Conversely, o1-mini shows less

TABLE III: Performance of Decompilation Methods across Various Obfuscation Techniques

Model	Bogus CF			CF Flattening			Ins. Substitution			BB Split		
	RE	RC	TCP	RE	RC	TCP	RE	RC	TCP	RE	RC	TCP
SecDec-6.7B	18.9	39.6	24.4	17.1	52.0	22.0	35.7	88.1	48.7	37.3	88.6	50.9
Nova-6.7B	9.9	74.4	22.5	6.4	64.5	15.5	9.8	77.4	24.9	9.8	76.8	25.0
LLM4Decompile-6.7B	17.8	39.8	22.8	16.2	44.7	20.3	32.0	81.7	45.3	33.2	83.7	45.4
ALT4Decompile-6.7B	25.5	84.9	36.5	20.3	85.1	30.7	40.1	95.1	54.3	38.3	95.3	53.8

suitability for decompilation tasks. Meanwhile, commercial decompilers and SAILR perform comparatively weaker on these metrics, as they are primarily designed to prioritize human readability over code re-execution, though established tools like Hex-Rays still maintain competitiveness. Hex-Rays occasionally introduces extra statements for readability, which may explain their lower RE compared to RetDec.

For refinement-based models, LLM4Decompile-Ref achieves a competitive 52.3% RE on Decompile-Eval (ranking second only to ALT4Decompile), but degrades sharply on Exebench. As discussed in Sections V and I, this discrepancy stems from its heavy reliance on the underlying decompiler's quality. For instance, Ghidra generates a mere 3.2% RE on Exebench, causing irreversible loss of semantic and structural information that strictly bounds the LLM's refinement capability. By contrast, ALT4Decompile's end-to-end processing of Assembly code completely bypasses this intermediate information loss. Moreover, although other dedicated LLMs perform decently, ALT4Decompile outpaces them by addressing their common inability to model high-level structures, leveraging its novel ALT reconstruction directly from low-level operations.

Answer to RQ1. ALT4Decompile demonstrates superior performance in decompilation compared to all baseline methods (even removing all refinement steps in Section III-D). For example, it achieves improvements of 4% in recompilation rate, 8.9% in re-execution rate, and 10.6% in test case pass rate over the SOTA End-to-End decompilation method on the Decompile-Eval dataset.

D. Performance under Code Obfuscation (RQ2)

To assess the robustness of ALT4Decompile, we evaluate its performance on binaries obfuscated with various methods. Code obfuscation is commonly employed in malware to conceal malicious behaviors, complicating analysis and detection. Following the previous work [51], we obfuscate source code from the Decompile-Eval using the widely adopted code obfuscation tool, Obfuscator-LLVM [37], and employ four standard obfuscation techniques: 1) *Bogus control flow* (BCF), which inserts fake control flows and pads with garbage instructions to change the program's control flow, 2) *Control flow flattening* (CFF), which reorganizes all basic blocks into a single level structure and encapsulates them within switch statements inside a loop, 3) *Instructions substitution* (IS), which replaces standard instructions with semantically equivalent but more complex instructions, and 4) *Basic block split* (BBS), which splits one basic block into multiple equivalent blocks.

Specifically, to ensure a fair comparison (e.g., input limit, model size and so on), we directly compare ALT4Decompile against all LLM-based end-to-end decompilation methods: SecDec, Nova, and LLM4Decompile. As shown in Table III, ALT4Decompile consistently outperforms all baseline methods across all obfuscation techniques, achieving an average improvement of 16.8% in RC rate, 3.8% in RE rate, and 7.3% in TCP rate. These results highlight the superior capability of ALT4Decompile in decompiling obfuscated binaries. Among the four obfuscation techniques, CFF and BCF exhibit the most significant impact on ALT4Decompile's performance, while IS has the least impact. This difference likely stems from the fact that CFF and BCF significantly obscure the underlying code logic by extensively altering the control flow, whereas IS mainly modifies individual instructions without affecting the overall program structure. Specifically, CFF affects performance by transforming loops into switch statements, indicating that incorporating additional logic node types (e.g., *Switch*) into the ALT could further enhance ALT4Decompile's effectiveness. Notably, ALT4Decompile demonstrates its greatest advantage over baselines in BCF, attributed to its ability to recover high-level structures and effectively distinguish fake control flows. In contrast, its performance in BBS obfuscation is comparatively modest, reflecting the challenges of reconstructing ALT after basic block splitting.

Answer to RQ2. ALT4Decompile achieves superior performance across four code obfuscation techniques, with CFF and BCF exerting the most substantial impact on the performance of all methods. It demonstrates its most significant advantage in BCF but a smaller one in BBS.

E. Ablation Study (RQ3)

We conduct five ablations to evaluate our designs: 1) ALT's structural advantage (Section IV-E1), 2) impact of instruction normalization (Section IV-E2), 3) other ALT4Decompile components (Section IV-E3), 4) fixers with different LLMs (Section IV-E4), and 5) ALT's scalability across model sizes (Section IV-E5).

1) *Structural Advantage of ALT*: We evaluate three variants (+Linear, +CFG [44], +ALT) using DeepSeekCoder and LLM4Decompile. All share the same training data and preprocessing pipeline to strictly control fair comparisons. Crucially, because all variants are identically equipped with IN, any performance gap is derived from different binary representation. Table IV shows +ALT (e.g., 57.2% RE on LLM4Decompile-6.7B) significantly outperforms +Linear (46.7% RE) and

TABLE IV: Effectiveness of ALT Compared with Different Binary Representations

Model	Re-execution Rate					Re-compilation Rate					TCP Rate
	O0	O1	O2	O3	Avg.	O0	O1	O2	O3	Avg.	
DeepSeekCoder-6.7B	0.0	0.0	0.0	0.0	0.0	1.2	0.0	0.0	1.8	0.8	0.0
+ Linear*	51.2	23.8	28.0	26.8	32.5	89.8	79.9	78.5	80.2	82.1	43.2
+ CFG*	52.4	24.2	28.1	26.6	32.8	90.1	79.1	78.7	80.5	82.1	44.7
+ ALT*	57.9	30.5	31.1	31.1	37.7	90.2	79.9	79.3	81.1	82.6	49.4
LLM4Decompile-6.7B	67.7	41.5	40.9	36.6	46.7	92.7	85.4	87.2	86.6	88.0	57.9
+ Linear*	66.5	48.2	45.7	41.5	50.5	92.1	88.6	90.2	89.6	90.1	60.4
+ CFG*	70.1	49.3	46.2	41.6	51.8	93.3	88.3	90.4	89.9	90.5	62.1
+ ALT*	82.3	51.8	48.8	45.7	57.2	95.7	88.4	90.9	90.2	91.3	67.1
+ ALT (w/o IN)	68.9	49.4	47.0	43.9	52.3	95.7	88.3	90.8	90.2	91.2	64.6
LLM4Decompile-1.3B	47.6	25.6	23.8	20.7	29.4	87.5	82.9	72.3	72.3	78.8	42.5
+ ALT*	62.8	33.5	28.1	28.0	38.1	89.6	84.1	76.2	75.6	81.4	50.1

* All fine-tuned variants include IN for fair comparison. +CFG follows the method in CodeInverter [44].

TABLE V: Ablation Results on VR, CEF, and BEF of ALT4Decompile

Model	Re-execution Rate					Re-compilation Rate					TCP Rate
	O0	O1	O2	O3	Avg.	O0	O1	O2	O3	Avg.	
ALT4Decompile	80.5	52.4	53.7	48.2	58.7	98.2	97.6	97.0	94.5	96.8	70.4
– w/o VR	76.5	49.9	50.4	44.8	55.4	96.3	96.1	94.7	91.2	94.6	65.6
– w/o BEF	82.9	52.4	50.0	47.6	58.2	98.2	97.6	97.0	95.1	97.0	69.2
– w/o CEF	78.7	51.8	56.7	47.6	58.7	95.7	92.7	95.1	93.3	94.2	70.1

VR: variable renaming; BEF: boundary error fixer; CEF: compilation error fixer.

+CFG (51.8% RE) across all bases. This compellingly confirms that ALT’s loop-oriented structure provides an obvious advantage over simple instruction-level operations, effectively guiding LLMs via shared logic patterns (Section II). Conversely, CFG models show inconsistent gains (sometimes underperforming +Linear), due to the LLMs’ limited capacity to infer high-level loops from raw control flows and limited training data.

2) *Impact of Instruction Normalization (IN)*: We test the variant + ALT (w/o IN) to explicitly evaluate IN’s contribution compared to + ALT (also means the performance of ALT4EXE model). Removing IN degrades the model’s RE from 57.2% to 52.3%, validating its role in supplying LLMs with crucial data references like embedded constants (Section II). Notably, despite lacking normalization, the unoptimized + ALT (w/o IN) still decisively beats the IN-equipped +Linear (50.5%) and +CFG (51.8%), reiterating the sheer dominance of ALT’s structural superiority.

3) *Impact of VR, CEF and BEF*: As shown in Table V, excluding variable renaming (VR) during the training stage drops the RE from 58.7% to 55.4%, validating our strategy of prioritizing execution semantics over symbol optimizations [32]. Similarly, removing the compilation error fixer (CEF) predictably degrades the RC. Interestingly, under O2 optimization, CEF slightly reduces RE; our analysis reveals that in these rare compiler-induced cases, the boundary error fixer (BEF) handles repairs more effectively. Overall, ALT remains independently powerful and complementary to these modular enhancements.

4) *Impact of different Fixer*: We evaluate the impact of different fixers implemented using various LLMs. Specifically, we select three representative LLMs and present the results

TABLE VI: Ablation Results on Fixers with Different LLMs

Model	RE (Avg.)	RC (Avg.)	TCP Rate
ALT4Decompile	58.7	96.8	70.4
w/ o1-mini	59.9	96.0	70.5
w/ GPT-4o	58.1	96.3	69.5
w/ Claude-3.5	60.8	96.8	72.2

in Table VI. It can be observed that Claude fixer achieves the highest overall performance. However, due to API usage cost consideration, we opt to use a more affordable model as the default fixer in ALT4Decompile. Although o1-mini shows unsatisfactory performance in decompilation tasks, it ranks second in decompilation code fixing. This observation indirectly highlights the importance of prioritizing the accurate recovery of execution semantics during decompilation.

5) *Performance across different Model Scale*: We further assess whether ALT maintains its advantages on smaller-scale models. As indicated in Table IV, ALT brings a significant improvement to both 1.3B and 6.7B models. Notably, the re-execution rate increases by 8.7% for the 1.3B model (from 29.4% to 38.1%) and by 10.5% for the 6.7B model (from 46.7% to 57.2%). These results confirm that ALT remains highly effective across varying model scales.

Answer to RQ3. Each component in ALT4Decompile contributes significantly to its overall performance, with ALT itself proving instrumental in enhancing decompilation capabilities across diverse model types and scales.

F. Real-world Applications (RQ4)

We compare ALT4Decompile against LLM-based end-to-end baselines on real-world software to ensure a fair com-

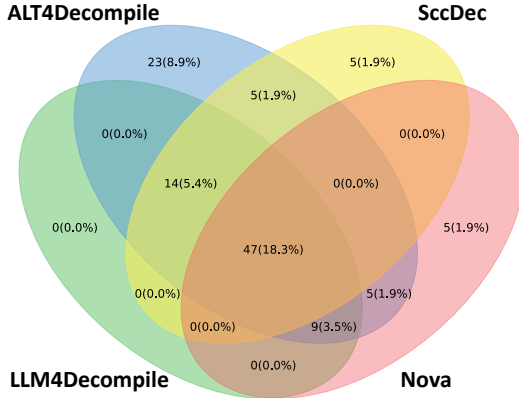


Fig. 4: Real-world Applications.

parison (e.g., input limit, model size and so on). Since real-world binaries lack test I/O pairs, we use an Assembly-search dataset [25] comprising 257 binary files (e.g., *WeChatWin.dll*) paired with natural language descriptions. After decompiling all functions, we use DeepSeek-V3 to summarize the outputs. Human evaluators then assess accuracy by comparing these summaries and the original Assembly against ground-truth descriptions. As Figure 4 shows, ALT4Decompile correctly decompiles the most functions (102, 39.7%), while Nova performs worst (66, 25.7%). Furthermore, 47 cases are solved by all methods, but ALT4Decompile achieves the most unique successes (23 cases).

Error Analysis. We identify three primary failure causes. First, all models frequently generate meaningless assignments (highest in SccDec at 48.9%, lowest in Nova at 34.0%). We attribute this to imbalanced training data in the base LLM4Decompile, which overemphasizes assignment patterns from Exebench (Section IV-A1). While our data filtering and Nova’s diverse training mitigated this, the bias persists. Second, the typical 4096-token context window of LLM-based methods truncates the decompilation of long real-world functions, causing incomplete outputs across all models. Finally, obfuscation triggers analysis tool failures (e.g., disassembly errors).

Answer to RQ4. ALT4Decompile achieves the highest accuracy (39.7%, a 12.1% improvement) on real-world software. However, inherent LLM context limits and analysis tool dependencies remain key bottlenecks limiting further gains.

G. Cost Study (RQ5)

In this RQ, we evaluate how much time and money ALT4Decompile and other methods cost on the Decompile-Eval dataset. We select the best-performing baselines (Claude, RetDec, SccDec) from the three baseline categories for comparison. The result is shown in Figure 5. Specifically, the decompilation time of ALT4Decompile is only 7 seconds longer than that of SccDec, with the additional time attributed to CFG extraction and ALT construction. Moreover, ALT4Decompile incurs an additional time cost of 17 seconds for the fixer and

symbolic information recovery phases. Among all methods, Claude is the slowest in getting feedback. In terms of monetary cost, ALT4Decompile incurs only \$0.49, which is significantly lower than Claude’s \$3.74. Notably, all of ALT4Decompile’s cost is attributed to symbolic information recovery and a small portion of error correction. As shown in Table IV, even without fixers and symbolic information recovery, ALT achieves a RE rate of 0.572 (only 0.015 lower than 0.587) on the LLM4Decompile model, which is still much higher than the other baseline models. Therefore, the additional \$0.49 and 17s can be considered optional when resources are constrained, allowing users to make a practical trade-off between performance and cost.

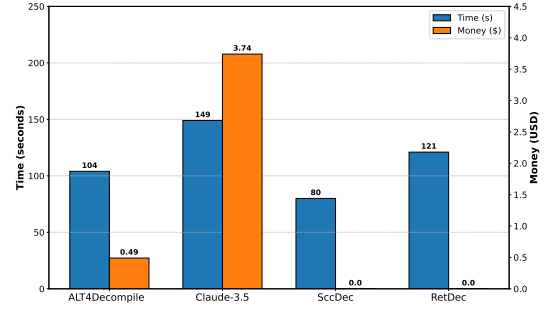


Fig. 5: Time and money cost of different methods.

Answer to RQ5. ALT4Decompile does not bring about excessive additional costs. The necessary additional time costs are CFG extraction and ALT construction (7s on the Decompile-Eval dataset).

H. User Study

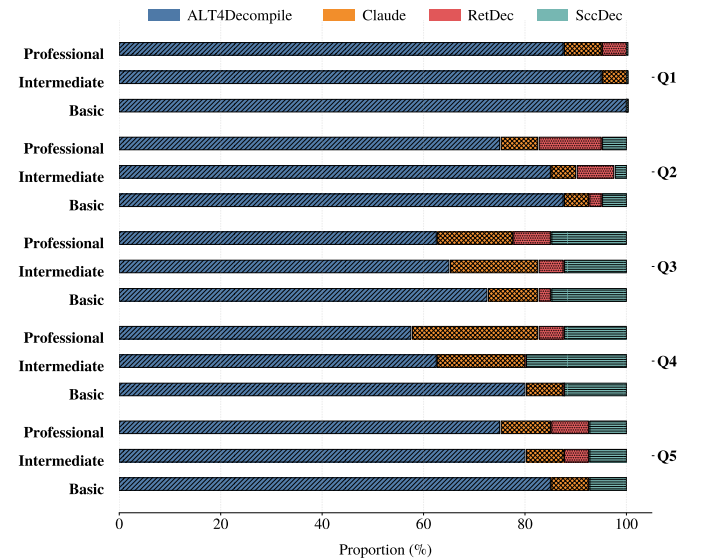


Fig. 6: User Study.

ALT4Decompile aims to produce decompiled code that (1) enables accurate program comprehension by reverse engineers, (2) preserves functional correctness for direct execution, and

(3) minimizes the effort required for practical reuse. Drawing from previous work [32], we conduct a user study comprising five questions to assess ALT4Decompile’s effectiveness in achieving this goal. Specifically, we recruit 12 participants, grouped by experience level: Professional (with over four years of reverse engineering experience), Intermediate (with 1-2 years of reverse engineering experience), and Basic (with an understanding of basic reverse engineering concepts and familiarity with code debugging processes). We compare ALT4Decompile against the top-performing baselines from three categories: Claude, RetDec, and SccDec. Each participant is presented with 10 randomly selected functions, along with the corresponding source code, test cases, and four decompiled outputs, anonymized to prevent bias. For each function, participants answer five evaluation questions. In total, we collect 120 responses per question. The questions are as follows:

- **Q1:** Which decompiled code contains the most meaningful and helpful comments?
- **Q2:** Which decompiled code has the most meaningful and helpful variable names?
- **Q3:** Which decompiled code best preserves the original function’s semantics or behavior?
- **Q4:** Which decompiled code requires the fewest debugging steps to pass all provided test cases?
- **Q5:** Overall, which decompiled code helps you best understand the function’s logic, compared to the original version?

As Figure 6 shows, ALT4Decompile consistently receives more positive feedback across all five questions. Notably, the Basic group found some original source codes difficult to comprehend but achieved better understanding using ALT4Decompile’s outputs. Conversely, consistent with DeGPT’s findings, the Professional group provided comparatively lower ratings; their extensive expertise naturally reduces reliance on LLM-generated readability enhancements, allowing them to effectively interpret raw commercial decompiler outputs. Furthermore, ALT4Decompile’s highest ratings in Q4 confirm its superior semantic preservation, directly corroborating our quantitative execution results in Section IV-C.

Answer to User Study. Overall, ALT4Decompile receives the most votes from the 12 participants, with the Basic group benefiting the most from our method. In addition, most participants agree that even if ALT4Decompile produced incorrect results, they could be corrected with minimal debugging effort.

V. RELATED WORK

Traditional decompilers [8] rely on program analysis, while recent advances leverage neural translation [55] to map binaries to high-level code. We categorize existing techniques into three groups:

Rule-based methods. Tools like Hex-Rays [8] and Ghidra [6] use static analysis and heuristics for type recovery and expression simplification. Meanwhile, Phoenix [15], DREAM [65], and SAILR [13] employ structuring algorithms to eliminate `gotos`. Despite their utility, they rely heavily on costly

expert knowledge and manual intervention. To address this, we systematically extract shared logic features between high- and low-level languages with minimal human effort, guiding LLMs toward more accurate decompilation.

Refine-based methods. These approaches use LLMs to optimize decompiler outputs. Examples include improving variable renaming and type prediction (DIRE [41], DIRTY [17], ReSym [61]), or enhancing readability and recompilability via error feedback mechanisms (DecGPT [59], DeGPT [32], DecLLM [60]). However, they rely heavily on the quality of the decompiler outputs, neglecting the overhead of decompiler extensions and the unverified efficacy of the models on raw binary files. Consequently, this paper focuses on Assembly code as the main research subject.

End-to-end methods. These methods directly utilize a translation model to decompile Assembly code. TRAFIX [38] and BTC [31] employ RNNs and Transformers to improve accuracy. Slade [12] utilizes PsycheC [45] to infer and supplement missing types. With the rise of LLMs, Nova [36] and LLM4Decompile [51] perform end-to-end training on large-scale source-Assembly pairs. SccDec [24] introduces a fine-grained alignment enhancement method (FAE) to improve LLM fine-tuning, thus improving the decompilation capabilities of LLMs. CodeInverter [44] introduces control flow graphs in language model training with millions of functions to improve decompilation. However, these methods still simply treat Assembly code as a new high-level language to process and attempt to solely rely on the strong learning ability of LLMs to complete decompilation, thereby resulting in no substantial performance gains with limited training (e.g., +CFG in experiments (Section IV-E1)).

In this paper, inspired by prior research [19], [35], [48], we observe that some logic patterns shared between high- (C) and low-level (Assembly) languages can effectively guide the decompilation LLM. Therefore, we propose a novel method for abstracting the high-level loop tree (ALT) from Assembly code, which is different from the existing methods that treat Assembly as a simple language sequence. Our ALT significantly improves the performance of LLM-based decompilation. Notably, unlike Wadec [50], which performs step-by-step decompilation by directly slicing the inherent loops in Wasm, we refactor C Assembly (which lacks logic structures such as loops) into ALT based on specific jump patterns, and inject it into the LLM all at once for decompilation.

VI. DISCUSSION

Applicable scenarios. Our ALT introduces a high-level loop framework inferred from shared logic patterns between high- (C) and low-level (Assembly) languages. However, this advantage diminishes when applied to functions comprising only simple assignments, as such functions generally pose less challenge for language models. In contrast, our method remains more effective in handling complex functions. The benefit may also decrease if certain analysis tools (e.g., disassemblers) or algorithms for identifying jumping units fail (e.g., wrong entry/exit nodes in connected subgraphs).

Future work. We explore the effectiveness of selecting the loop structure as a ALT node, which brings significant decompilation LLM performance improvement. It dues to its significant impact on decompilation failures of existing LLM-based methods (shown in Section II). Future work will incorporate other high-level structures such as conditional statements (*if*) to enhance ALT’s robustness against diverse code obfuscations and optimizations.

Selected evaluation metrics. Previous methods often relied on similarity metrics (e.g., edit distance) to evaluate decompilation quality. However, these metrics have notable limitations. First, a low similarity score does not necessarily indicate incorrectness, as the same functionality can be implemented using different logic structures (e.g., a *for* loop can be replaced by a *while* loop). Second, a high similarity score does not guarantee correctness, since even a minor change (e.g., flipping the comparison operator $>$ to $<$ in an *if* node) can alter the entire execution path of the program. As discussed in Section I, this paper focuses on restoring the execution logic correctness of source code. Therefore, we evaluate the functional correctness of the decompiled results through test case execution (RE and TCP), ensuring that the decompiled code is not only logically correct but also directly reusable. These metrics provide a more rigorous and meaningful assessment of decompilation quality than traditional superficial similarity-based metrics.

VII. CONCLUSION

In this paper, we propose ALT4Decompile, an LLM-based binary decompilation technique. Unlike previous methods that directly process Assembly as another high-level language, we infer a ALT from binaries, thereby mitigating the syntactic gap between high- and low-level code. Evaluation on three datasets demonstrates the effectiveness of ALT4Decompile, i.e., showing a 4% improvement in re-compilation rate, an 8.9% improvement in re-execution rate, and a 10.6% improvement in test case pass rate compared to SOTA methods on the Decompile-Eval. We also conduct a real-world software analysis and user study to further evaluate ALT4Decompile.

REFERENCES

- [1] “Angr,” <https://github.com/angr/angr>, 2026.
- [2] “anonymous,” <https://doi.org/10.5281/zenodo.19247495>, 2026.
- [3] “Clang-format,” <https://clang.llvm.org/docs/ClangFormat.html>, 2026.
- [4] “Claude-3.5-sonnet,” <https://www.anthropic.com/news/claude-3-5-sonnet>, 2026.
- [5] “Gcc compiler,” <https://gcc.gnu.org/>, 2026.
- [6] “Ghidra,” <https://github.com/NationalSecurityAgency/ghidra>, 2026.
- [7] “Gpt-4o,” <https://openai.com/index/hello-gpt-4o/>, 2026.
- [8] “Ida pro,” <https://hex-rays.com/IDA-pro/>, 2026.
- [9] “O1-mini,” <https://openai.com/index/openai-o1-mini-advancing-cost-efficient-reasoning/>, 2026.
- [10] “Tree-itter,” <https://tree-sitter.github.io/tree-sitter/>, 2026.
- [11] J. Armengol-Estapé, J. Woodruff, A. Brauckmann, J. W. d. S. Magalhães, and M. F. O’Boyle, “Exebench: an ml-scale dataset of executable c functions,” in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, 2022, pp. 50–59.
- [12] J. Armengol-Estapé, J. Woodruff, C. Cummins, and M. F. O’Boyle, “Slade: A portable small language model decompiler for optimized assembly,” in *2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2024, pp. 67–80.
- [13] Z. L. Basque, A. P. Bajaj, W. Gibbs, J. O’Kain, D. Miao, T. Bao, A. Doupe, Y. Shoshitaishvili, and R. Wang, “Ahoy sailor! there is no need to DREAM of C: A compiler-aware structuring algorithm for binary decompilation,” in *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*, D. Balzarotti and W. Xu, Eds. USENIX Association, 2024. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/basque>
- [14] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, “Directed greybox fuzzing,” in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 2329–2344.
- [15] D. Brumley, J. Lee, E. J. Schwartz, and M. Woo, “Native x86 decompilation using {Semantics-Preserving} structural analysis and iterative {Control-Flow} structuring,” in *22nd USENIX Security Symposium (USENIX Security 13)*, 2013, pp. 353–368.
- [16] K. Burk, F. Pagani, C. Kruegel, and G. Vigna, “Decompersion: How humans decompile and what we can learn from it,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 2765–2782.
- [17] Q. Chen, J. Lacomis, E. J. Schwartz, C. Le Goues, G. Neubig, and B. Vasilescu, “Augmenting decompiler output with learned variable names and types,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 4327–4343.
- [18] M. Christodorescu, S. Jha, S. A. Seshia, D. Song, and R. E. Bryant, “Semantics-aware malware detection,” in *2005 IEEE symposium on security and privacy (S&P’05)*. IEEE, 2005, pp. 32–46.
- [19] C. Cifuentes and M. Van Emmerik, “Recovery of jump table case statements from binary code,” in *Proceedings Seventh International Workshop on Program Comprehension*, 1999, pp. 192–199.
- [20] S. Dinesh, N. Burrow, D. Xu, and M. Payer, “Retrowrite: Statically instrumenting cots binaries for fuzzing and sanitization,” in *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2020, pp. 1497–1511.
- [21] S. H. Ding, B. C. Fung, and P. Charland, “Asm2vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization,” in *2019 IEEE symposium on security and privacy (sp)*. IEEE, 2019, pp. 472–489.
- [22] L. Dramko, J. Lacomis, E. J. Schwartz, B. Vasilescu, and C. Le Goues, “A taxonomy of c decompiler fidelity issues,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 379–396.
- [23] H. Eom, D. Kim, S. Lim, H. Koo, and S. Hwang, “R2i: A relative readability metric for decompiled code,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 383–405, 2024.
- [24] Y. Feng, D. Teng, Y. Xu, H. Mu, X. Xu, L. Qin, Q. Zhu, and W. Che, “Self-constructed context decompilation with fined-grained alignment enhancement,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 6603–6614.
- [25] Z. Gao, H. Wang, Y. Wang, and C. Zhang, “Virtual compiler is all you need for assembly code search,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 3040–3051.
- [26] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, “Deepseek-coder: When the large language model meets programming—the rise of code intelligence,” *arXiv preprint arXiv:2401.14196*, 2024.
- [27] I. U. Haq and J. Caballero, “A survey of binary code similarity,” *Acm computing surveys (csur)*, vol. 54, no. 3, pp. 1–38, 2021.
- [28] H. He, X. Lin, Z. Weng, R. Zhao, S. Gan, L. Chen, Y. Ji, J. Wang, and Z. Xue, “Code is not natural language: Unlock the power of semantics-oriented graph representation for binary code similarity detection,” in *33rd USENIX Security Symposium (USENIX Security 24)*, PHILADELPHIA, PA, 2024.
- [29] S. Heelan and A. Gianni, “Augmenting vulnerability analysis of binary code,” in *Proceedings of the 28th Annual Computer Security Applications Conference*, 2012, pp. 199–208.
- [30] Hex-Rays SA, “Ida pro – hex rays,” <https://www.hex-rays.com/products/ida/>, 2020, accessed: 2026-01-29.
- [31] I. Hosseini and B. Dolan-Gavitt, “Beyond the c: Retargetable decompilation using neural machine translation,” *arXiv preprint arXiv:2212.08950*, 2022.
- [32] P. Hu, R. Liang, and K. Chen, “Degpt: Optimizing decompiler output with llm,” in *Proceedings 2024 Network and Distributed System Security Symposium*, vol. 267622140, 2024.
- [33] Y. Hu, Y. Zhang, J. Li, and D. Gu, “Binary code clone detection across architectures and compiling configurations,” in *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*. IEEE, 2017, pp. 88–98.
- [34] N. Idika and A. P. Mathur, “A survey of malware detection techniques,” *Purdue University*, vol. 48, no. 2, pp. 32–46, 2007.

- [35] L. Jiang, J. An, H. Huang, Q. Tang, S. Nie, S. Wu, and Y. Zhang, "Binaryai: binary software composition analysis via intelligent binary source code matching," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [36] N. Jiang, C. Wang, K. Liu, X. Xu, L. Tan, X. Zhang, and P. Babkin, "Nova: Generative language models for assembly code with hierarchical attention and contrastive learning," in *ICLR*, 2025.
- [37] P. Junod, J. Rinaldini, J. Wehrli, and J. Michielin, "Obfuscator-llvm—software protection for the masses," in *2015 IEEE/ACM 1st international workshop on software protection*. IEEE, 2015, pp. 3–9.
- [38] O. Katz, Y. Olshaker, Y. Goldberg, and E. Yahav, "Towards neural decompilation," *arXiv preprint arXiv:1905.08325*, 2019.
- [39] V. Kovah, "Finding new bluetooth low energy exploits via reverse engineering multiple vendors' firmwares," *DEF CON*, vol. 7, no. 8, 2020.
- [40] J. Křoustek, P. Matula, and P. Zemek, "Retdec: An open-source machine-code decompiler," in *Botconf 2017*, Montpellier, France, 2017.
- [41] J. Lacomis, P. Yin, E. Schwartz, M. Allamanis, C. Le Goues, G. Neubig, and B. Vasilescu, "Dire: A neural approach to decompiled identifier naming," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 628–639.
- [42] S. Li, Y. Zheng, H. Li, J. Guo, C. Dong, C. Yan, W. Wang, Y. Ren, L. Sun, and H. Zhu, "Lares: Llm-driven code slice semantic search for patch presence testing," *arXiv preprint arXiv:2511.01252*, 2025.
- [43] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.
- [44] P. Liu, J. Sun, L. Chen, Z. Yan, P. Zhang, D. Sun, D. Wang, and D. Li, "Control flow-augmented decompiler based on large language model," *arXiv preprint arXiv:2503.07215*, 2025.
- [45] L. T. Melo, R. G. Ribeiro, M. R. de Araújo, and F. M. Q. Pereira, "Inference of static semantics for incomplete c programs," *Proceedings of the ACM on Programming Languages*, vol. 2, no. POPL, pp. 1–28, 2017.
- [46] S. Nagy, A. Nguyen-Tuong, J. D. Hiser, J. W. Davidson, and M. Hicks, "Breaking through binaries: Compiler-quality instrumentation for better binary-only fuzzing," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 1683–1700.
- [47] C. Pang, R. Yu, Y. Chen, E. Koskinen, G. Portokalidis, B. Mao, and J. Xu, "Sok: All you ever wanted to know about x86/x64 binary disassembly but were afraid to ask," in *2021 IEEE symposium on security and privacy (SP)*. IEEE, 2021, pp. 833–851.
- [48] J. Powny, F. Schuster, L. Bernhard, T. Holz, and C. Rossow, "Leveraging semantic signatures for bug search in binary programs," in *Proceedings of the 30th Annual Computer Security Applications Conference*, ser. ACSAC '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 406–415. [Online]. Available: <https://doi.org/10.1145/2664243.2664269>
- [49] P. Reiter, H. J. Tay, W. Weimer, A. Doupé, R. Wang, and S. Forrest, "Automatically mitigating vulnerabilities in binary programs via partially recompilable decompilation," *IEEE Transactions on Dependable and Secure Computing*, 2024.
- [50] X. She, Y. Zhao, and H. Wang, "Wadec: Decompiling webassembly using large language model," in *Proceedings of the 39th IEEE/ACM international conference on automated software engineering*, 2024, pp. 481–492.
- [51] H. Tan, Q. Luo, J. Li, and Y. Zhang, "Llm4decompile: Decompiling binary code with large language models," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 3473–3487.
- [52] H. Tan, X. Tian, H. Qi, J. Liu, Z. Gao, S. Wang, Q. Luo, J. Li, and Y. Zhang, "Decompile-bench: Million-scale binary-source function pairs for real-world binary decompilation," 2025. [Online]. Available: <https://arxiv.org/abs/2505.12668>
- [53] W. Tang, P. Luo, J. Fu, and D. Zhang, "Libdx: A cross-platform and accurate system to detect third-party libraries in binary code," in *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2020, pp. 104–115.
- [54] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM journal on computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [55] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [56] F. Verbeek, P. Olivier, and B. Ravindran, "Sound c code decompilation for a subset of x86-64 binaries," in *International Conference on Software Engineering and Formal Methods*. Springer, 2020, pp. 247–264.
- [57] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, "Jtrans: Jump-aware transformer for binary code similarity detection," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2022, pp. 1–13.
- [58] R. Wang, Y. Shoshitaishvili, A. Bianchi, A. Machiry, J. Grosen, P. Grosen, C. Kruegel, and G. Vigna, "Ramblr: Making reassembly great again," in *NDSS*, 2017.
- [59] W. K. Wong, H. Wang, Z. Li, Z. Liu, S. Wang, Q. Tang, S. Nie, and S. Wu, "Refining decompiled c code with large language models," *arXiv preprint arXiv:2310.06530*, 2023.
- [60] W. K. Wong, D. Wu, H. Wang, Z. Li, Z. Liu, S. Wang, Q. Tang, S. Nie, and S. Wu, "Decllm: Llm-augmented recompilable decompilation for enabling programmatic use of decompiled code," *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1841–1864, 2025.
- [61] D. Xie, Z. Zhang, N. Jiang, X. Xu, L. Tan, and X. Zhang, "Resym: Harnessing llms to recover variable and data structure symbols from stripped binaries," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 4554–4568.
- [62] X. Xu, Q. Zheng, Z. Yan, M. Fan, A. Jia, and T. Liu, "Interpretation-enabled software reuse detection based on a multi-level birthmark model," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 873–884.
- [63] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, "Neural network-based graph embedding for cross-platform binary code similarity detection," in *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, 2017, pp. 363–376.
- [64] K. Yakdan, S. Dechand, E. Gerhards-Padilla, and M. Smith, "Helping johnny to analyze malware: A usability-optimized decompiler and malware analysis user study," in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 158–177.
- [65] K. Yakdan, S. Eschweiler, E. Gerhards-Padilla, and M. Smith, "No more gotos: Decompilation using pattern-independent control-flow structuring and semantic-preserving transformations," in *NDSS*, 2015.
- [66] Z. Yuan, M. Feng, F. Li, G. Ban, Y. Xiao, S. Wang, Q. Tang, H. Su, C. Yu, J. Xu *et al.*, "B2sfinder: Detecting open-source software reuse in cots software," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 1038–1049.
- [67] Z. Zhou, X. Li, R. Feng, Y. Zhang, Y. Li, W. Feng, Y. Wang, and Y. Li, "Fidelitygpt: Correcting decompilation distortions with retrieval augmented generation," *arXiv preprint arXiv:2510.19615*, 2025.